

Interaktive Lernplattform für Grundlagen Informatik

Projektarbeit – SS 26

17. Juni 2026

Studenten:	Markus Nöth	markus.noeth@study.thws.de	5123027
	Richard Schröder	richard.schroeder@study.thws.de	5123042
	Enrico Ebert	enrico.ebert@study.thws.de	5123098
	Duy Tan Pham	duytan.pham@study.thws.de	5123063
	Kristian Popov	kristian.popov@study.thws.de	5123029

Dozent: Prof. Dr. Peter Braun

Inhaltsverzeichnis

1 Einleitung **1**
1.1 Problemstellung (Author) . . . 1
1.2 Zielsetzung der Arbeit (Author) . . . 1
1.3 Methodik (Author) . . . 1

2 Übergreifende Architektur & Systemdesign **1**
2.1 Technologiestack (Author) . . . 1
2.2 Modulare Softwarearchitektur (Author) . . . 1
2.3 Unified UI-Konzept (Author) . . . 1
2.4 Simulator-Architektur (Author) . . . 1
2.5 Datenmodellierung (Author) . . . 1

3 Einsatz von Künstlicher Intelligenz im Projekt **1**
3.1 KI Reflexion (Markus Nöth) . . . 1
3.1.1 KI als Co-Developer (Markus Nöth) . . . 2
3.1.2 Automatisierte Aufgabengenerierung (Markus Nöth) . . . 2
3.1.3 Kritische Reflexion (Markus Nöth) . . . 2
3.2 KI Reflexion (Richard Schröder) . . . 2
3.2.1 KI als Co-Developer (Richard Schröder) . . . 2
3.2.2 Automatisierte Aufgabengenerierung (Richard Schröder) . . . 3
3.2.3 Kritische Reflexion (Richard Schröder) . . . 3
3.3 KI Reflexion (Enrico Ebert) . . . 4
3.3.1 KI als Co-Developer (Enrico Ebert) . . . 4
3.3.2 Automatisierte Aufgabengenerierung (Enrico Ebert) . . . 4
3.3.3 Kritische Reflexion (Enrico Ebert) . . . 4
3.4 KI Reflexion (Duy Tan Pham) . . . 4
3.4.1 KI als Co-Developer (Duy Tan Pham) . . . 4
3.4.2 Automatisierte Aufgabengenerierung (Duy Tan Pham) . . . 4
3.4.3 Kritische Reflexion (Duy Tan Pham) . . . 4
3.5 KI Reflexion (Kristian Popov) . . . 4
3.5.1 KI als Co-Developer (Kristian Popov) . . . 4
3.5.2 Automatisierte Aufgabengenerierung (Kristian Popov) . . . 4
3.5.3 Kritische Reflexion (Kristian Popov) . . . 4

4 Designentscheidungen der Fachmodule **5**
4.1 Turing Maschine (Author) . . . 5
4.1.1 Entscheidungen in der Architektur (Author) . . . 5
4.1.2 Entscheidungen im UI (Author) . . . 5
4.1.3 Andere Entscheidungen (Author) . . . 5
4.2 Endliche Automaten (Author) . . . 5
4.2.1 Entscheidungen in der Architektur (Author) . . . 5
4.2.2 Entscheidungen im UI (Author) . . . 5
4.2.3 Andere Entscheidungen (Author) . . . 5
4.3 Reguläre Sprachen (Author) . . . 5
4.3.1 Entscheidungen in der Architektur (Author) . . . 5
4.3.2 Entscheidungen im UI (Author) . . . 5
4.3.3 Andere Entscheidungen (Author) . . . 5
4.4 Zahlensysteme (Author) . . . 5
4.4.1 Entscheidungen in der Architektur (Author) . . . 5

4.4.2 Entscheidungen im UI (Author) . . . 5
4.4.3 Andere Entscheidungen (Author) . . . 5
4.5 Moore & Mealy Automaten (Author) . . . 5
4.5.1 Entscheidungen in der Architektur (Author) . . . 5
4.5.2 Entscheidungen im UI (Author) . . . 5
4.5.3 Andere Entscheidungen (Author) . . . 5
4.6 Disjunktive Normalform (Author) . . . 6
4.6.1 Entscheidungen in der Architektur (Author) . . . 6
4.6.2 Entscheidungen im UI (Author) . . . 6
4.6.3 Andere Entscheidungen (Author) . . . 6
4.7 Encoding (Author) . . . 6
4.7.1 Entscheidungen in der Architektur (Author) . . . 6
4.7.2 Entscheidungen im UI (Author) . . . 6
4.7.3 Andere Entscheidungen (Author) . . . 6
4.8 Java (Author) . . . 6
4.8.1 Entscheidungen in der Architektur (Author) . . . 6
4.8.2 Entscheidungen im UI (Author) . . . 6
4.8.3 Andere Entscheidungen (Author) . . . 6

5 Evaluation & Projekterfahrung 6
5.1 Qualitätssicherung (Author) . . . 6
5.2 Abgleich mit dem Lastenheft (Author) . . . 6
5.3 Lessons Learned (Author) . . . 6
5.4 Design-Iterationen (Author) . . . 6

6 Fazit & Ausblick 7
6.1 Zusammenfassung der Projektergebnisse (Author) . . . 7
6.2 Perspektiven (Author) . . . 7

Anhang i
Abkürzungsverzeichnis . . . i
Literatur . . . i

Abbildungsverzeichnis

Tabellenverzeichnis

1 Einleitung

Eine Arbeit von Technische Hochschule Würzburg-Schweinfurt (THWS) Studenten.

1.1 Problemstellung (Author)

Die Hürde der Abstraktion in Grundlagen Informatik

1.2 Zielsetzung der Arbeit (Author)

Didaktische Aufbereitung komplexer Modelle durch interaktive Simulationen

1.3 Methodik (Author)

Vorgehensweise im Team und Rollenverteilung

2 Übergreifende Architektur & Systemdesign

2.1 Technologiestack (Author)

React 19, TypeScript, Vite, Tailwind CSS 4, shadcn/ui, Zustand, i18next, Lucide Icons

2.2 Modulare Softwarearchitektur (Author)

Single Source of Truth (`moduleCatalog.tsx`), lazy Loading, Shared Topic Pattern – Sicherstellung der Erweiterbarkeit für neue Themen

2.3 Unified UI-Konzept (Author)

Wiederverwendbare Shared Components, konsistentes Designsystem (CSS-Variablen, Dark Mode, moduleigene Akzentfarben, Geist Font)

2.4 Simulator-Architektur (Author)

Das `ISimulator<TConfig, TState>`-Interface und der generische `useSimulationEngine`-Hook als Grundlage aller schrittweisen Simulationen

2.5 Datenmodellierung (Author)

Import/Export über `ISimulatorTransferCodec` mit Base64-Kodierung für Vorlesungskompatibilität

3 Einsatz von Künstlicher Intelligenz im Projekt

3.1 KI Reflexion (Markus Nöth)

Nutzung von Gemini 3.1 Pro, ChatGPT und Codex als Entwicklungswerkzeuge – Effizienzsteigerung bei der Implementierung mathematischer Logik und UI-Komponenten

3.1.1 KI als Co-Developer (Markus Nöth)

Reflektieren und so...

3.1.2 Automatisierte Aufgabengenerierung (Markus Nöth)

Reflektieren und so...

3.1.3 Kritische Reflexion (Markus Nöth)

Reflektieren und so...

3.2 KI Reflexion (Richard Schröder)

3.2.1 KI als Co-Developer (Richard Schröder)

Im Rahmen der Projektarbeit wurden verschiedene KI-gestützte Werkzeuge als unterstützende Systeme in der Softwareentwicklung eingesetzt. Dazu zählten insbesondere ChatGPT, Codex in den Entwicklungsumgebungen WebStorm und Zed sowie lokal ausgeführte Modelle über Ollama in Zed. Ergänzend wurde Gemini vereinzelt für Rechercheaufgaben genutzt, wenn zusätzliche externe Informationen oder alternative Formulierungen benötigt wurden.

Der Einsatz dieser Werkzeuge hat den Entwicklungsprozess deutlich erleichtert, da Entwürfe, Implementierungsvorschläge und Reviews wesentlich schneller erstellt werden konnten. Gleichzeitig zeigte sich jedoch, dass die generierten Ergebnisse nicht unkritisch übernommen werden dürfen. Eine fachliche Kontrolle blieb in allen Phasen notwendig, insbesondere bei Architekturentscheidungen, bei Änderungen an bestehendem Code und bei der Bewertung der Testabdeckung.

Das allgemeine Vorgehen beim Einsatz von KI als Co-Developer bestand aus mehreren wiederkehrenden Schritten:

1. Zunächst wurde der gewünschte Zielzustand des Systems möglichst detailliert beschrieben. Auf dieser Grundlage wurde ein erster Architekturvorschlag durch das KI-System erzeugt, der als Ausgangspunkt für die weitere Planung diente.
2. Anschließend wurde der Architekturvorschlag fachlich geprüft und überarbeitet. Dies war erforderlich, da die ersten Vorschläge teilweise zu allgemein waren oder sich nicht optimal in die bereits bestehende Systemarchitektur einfügten.
3. Der verbesserte Architekturentwurf wurde danach in konkrete Arbeitspakete zerlegt. Auch diese Aufteilung wurde kontrolliert, um sicherzustellen, dass die Aufgaben fachlich sinnvoll abgegrenzt waren und keine relevanten Abhängigkeiten übersehen wurden.
4. Die einzelnen Arbeitspakete wurden anschließend in separaten Prompts bearbeitet, wobei die Gesamtarchitektur weiterhin als Kontext mitgegeben wurde. Nach jeder Umsetzung wurde die generierte Implementierung manuell überprüft, um Fehler, unpassende Änderungen oder Abweichungen vom geplanten Vorgehen frühzeitig zu erkennen.
5. Zusätzlich wurde der generierte Code in einem neuen Chat erneut durch KI geprüft. Dabei lag der Fokus auf der Übereinstimmung mit dem zuvor definierten Architekturdokument sowie auf einem allgemeinen Code Review hinsichtlich Verständlichkeit, Struktur und möglicher Schwachstellen.
6. Besondere Aufmerksamkeit wurde auf korrekte und vollständige Tests gelegt. Neben Unit-Tests wurden auch End-to-End-Tests betrachtet, da diese für die Absicherung der

implementierten Nutzerabläufe und der Integration verschiedener Komponenten besonders relevant waren.

3.2.2 Automatisierte Aufgabengenerierung (Richard Schröder)

Ein weiteres Einsatzfeld der KI-Systeme war die automatisierte Generierung von Aufgaben. Zu Beginn waren die Ergebnisse nur eingeschränkt verwendbar, insbesondere bei Aufgaben zur Turingmaschine. Häufig wurde das geforderte JSON Format nicht korrekt eingehalten, und die erzeugten Lösungen zeigten, dass ChatGPT und Codex in diesem speziellen Themenbereich nicht ohne zusätzliche Vorgaben als zuverlässige Experten eingesetzt werden konnten.

Die Qualität der Ergebnisse verbesserte sich, nachdem mehrere korrekt aufgebaute Turingmaschinen erstellt und als Kontext für weitere Generierungen verwendet wurden. Dadurch konnten sich die Modelle besser an das gewünschte Format und die erwartete Aufgabenstruktur orientieren. Dennoch blieb weiterhin eine fachliche Kontrolle notwendig, da auch mit verbessertem Kontext nicht alle generierten Turingmaschinen vollständig korrekt waren.

Das Nachbauen von Aufgaben aus Skript und Übungsblättern war hingegen gut umsetzbar, wenn ausreichend JSON Beispiele als Kontext bereitgestellt wurden. Besonders hilfreich war dabei die Kombination aus bestehenden Aufgabenbeispielen und einem Screenshot der jeweiligen Lösung. Auf dieser Grundlage konnte ChatGPT vergleichsweise zuverlässig Lösungen erzeugen, die sich an Struktur und Formulierung der Vorlagen orientierten.

3.2.3 Kritische Reflexion (Richard Schröder)

Insgesamt stellte der Einsatz von KI im Projekt eine erhebliche Unterstützung dar, insbesondere um Entwicklungsaufgaben schneller zu strukturieren, Implementierungen vorzubereiten und alternative Lösungsansätze zu prüfen. Dadurch konnte effizienter gearbeitet werden, ohne jeden Entwurf oder jede technische Detailentscheidung vollständig von Grund auf manuell ausarbeiten zu müssen. Der Nutzen war jedoch besonders dann hoch, wenn die Anfragen klar eingegrenzt und die Ergebnisse anschließend überprüft wurden.

Eine zentrale Einschränkung bestand darin, dass zu große Aufgaben nicht zuverlässig in einem einzigen Schritt bearbeitet werden konnten. Wurde beispielsweise versucht, eine vollständige Turingmaschine mit allen Details auf einmal generieren zu lassen, konnten Teile der Anforderungen aus dem Kontext verloren gehen oder fehlerhaft umgesetzt werden. Daher erwies es sich als sinnvoller, komplexe Aufgaben in kleinere Einheiten zu zerlegen und diese nacheinander zu bearbeiten.

Darüber hinaus zeigte sich gelegentlich eine Tendenz der KI-Systeme, einfache Problemstellungen unnötig komplex zu lösen. In solchen Fällen wurden umfangreiche Strukturen, zusätzliche Abstraktionen oder technische Umwege vorgeschlagen, obwohl eine deutlich einfachere Lösung ausreichend gewesen wäre. Dies machte eine kritische Bewertung der vorgeschlagenen Architektur und Implementierung besonders wichtig.

Problematisch war außerdem, dass KI-generierte Änderungen nicht immer an der fachlich passenden Stelle vorgenommen wurden. Teilweise wurden allgemeine oder gemeinsam genutzte Komponenten angepasst, obwohl eine lokale Änderung innerhalb des jeweiligen Moduls angemessener gewesen wäre. Solche Änderungen können unerwünschte Auswirkungen auf andere Teile des Systems haben und verdeutlichen, dass KI-Unterstützung nur in Verbindung mit einem guten Verständnis der bestehenden Codebasis sinnvoll eingesetzt werden sollte.

3.3 KI Reflexion (Enrico Ebert)

Nutzung von Gemini 3.1 Pro, ChatGPT und Codex als Entwicklungswerkzeuge – Effizienzsteigerung bei der Implementierung mathematischer Logik und UI-Komponenten

3.3.1 KI als Co-Developer (Enrico Ebert)

Reflektieren und so...

3.3.2 Automatisierte Aufgabengenerierung (Enrico Ebert)

Reflektieren und so...

3.3.3 Kritische Reflexion (Enrico Ebert)

Reflektieren und so...

3.4 KI Reflexion (Duy Tan Pham)

Nutzung von Gemini 3.1 Pro, ChatGPT und Codex als Entwicklungswerkzeuge – Effizienzsteigerung bei der Implementierung mathematischer Logik und UI-Komponenten

3.4.1 KI als Co-Developer (Duy Tan Pham)

Reflektieren und so...

3.4.2 Automatisierte Aufgabengenerierung (Duy Tan Pham)

Reflektieren und so...

3.4.3 Kritische Reflexion (Duy Tan Pham)

Reflektieren und so...

3.5 KI Reflexion (Kristian Popov)

Nutzung von Gemini 3.1 Pro, ChatGPT und Codex als Entwicklungswerkzeuge – Effizienzsteigerung bei der Implementierung mathematischer Logik und UI-Komponenten

3.5.1 KI als Co-Developer (Kristian Popov)

Reflektieren und so...

3.5.2 Automatisierte Aufgabengenerierung (Kristian Popov)

Reflektieren und so...

3.5.3 Kritische Reflexion (Kristian Popov)

Reflektieren und so...

4 Designentscheidungen der Fachmodule

4.1 Turing Maschine (Author)

4.1.1 Entscheidungen in der Architektur (Author)

4.1.2 Entscheidungen im UI (Author)

4.1.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.2 Endliche Automaten (Author)

4.2.1 Entscheidungen in der Architektur (Author)

4.2.2 Entscheidungen im UI (Author)

4.2.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.3 Reguläre Sprachen (Author)

4.3.1 Entscheidungen in der Architektur (Author)

4.3.2 Entscheidungen im UI (Author)

4.3.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.4 Zahlensysteme (Author)

4.4.1 Entscheidungen in der Architektur (Author)

4.4.2 Entscheidungen im UI (Author)

4.4.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.5 Moore & Mealy Automaten (Author)

4.5.1 Entscheidungen in der Architektur (Author)

4.5.2 Entscheidungen im UI (Author)

4.5.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.6 Disjunktive Normalform (Author)

4.6.1 Entscheidungen in der Architektur (Author)

4.6.2 Entscheidungen im UI (Author)

4.6.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.7 Encoding (Author)

4.7.1 Entscheidungen in der Architektur (Author)

4.7.2 Entscheidungen im UI (Author)

4.7.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.8 Java (Author)

4.8.1 Entscheidungen in der Architektur (Author)

4.8.2 Entscheidungen im UI (Author)

4.8.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

5 Evaluation & Projekterfahrung

5.1 Qualitätssicherung (Author)

CI/CD-Pipeline (GitLab Runner auf Raspberry Pi), automatisiertes Linting, Formatierung (Prettier), Unit-Tests (Vitest) und E2E-Tests (Playwright)

5.2 Abgleich mit dem Lastenheft (Author)

Wurden die Qualitätsziele (Performance, Usability, Barrierefreiheit durch Dark Mode und i18n) erreicht?

5.3 Lessons Learned (Author)

Herausforderungen in der Team-Kollaboration (Git-Workflow, Merge-Requests, geteilte Architektur) und technische Hürden

5.4 Design-Iterationen (Author)

Welche Entscheidungen mussten im Prozess revidiert werden?

6 Fazit & Ausblick

6.1 Zusammenfassung der Projektergebnisse (Author)

Sieben interaktive Fachmodule in zwei Lernbereichen

6.2 Perspektiven (Author)

Gamification (Badges/Punkte), LMS-Anbindung (Moodle/ILIAS) und Mobile Learning (responsive Optimierung)

Anhang

Abkürzungsverzeichnis

THWS Technische Hochschule Würzburg-Schweinfurt; Hochschule aus Würzburg und Schweinfurt (Bayern) [1]

Literatur

[1] Technische Hochschule Würzburg-Schweinfurt. „THWS - Technische Hochschule Würzburg-Schweinfurt,“ besucht am 14. Juni 2026. Adresse: `https://thws.de`.